

Control Flow: Operators

 <https://github.com/learn-co-curriculum/phase-3-python-control-flow-operators>  <https://github.com/learn-co-curriculum/phase-3-python-control-flow-operators/issues/new>

Learning Goals

- Use common comparison functions for control flow (`==` , `!=` , `>` , `<`).
- Use common logical operators for control flow (`and` , `or` , `not`).
- Understand the differences in syntax between Python and JavaScript.

Key Vocab

- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell:** an interactive interpreter that can be accessed from the command line.
- **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception:** a type of error that can be predicted and handled without causing a program to crash.
- **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Function:** a named code block that performs a sequence of actions when it is called.
- **Scope:** the area in your program where a specific variable can be called.

Introduction

Just like JavaScript, Python has several ways we can control the flow of execution in our programs:

- We can use conditional statements like `if/else` and `try/except`
- We can use looping constructs like `for` and `while`

Using these control flow constructs means we're taking our code out of the normal flow of execution (top-to-bottom, one line at a time) and instead providing some instructions to change that order. As you've surely seen in JavaScript, conditional statements and loops are critical for writing applications.

In the next series of lessons, we'll explore common approaches to control flow, and learn some new syntax that is unique to Python.

Comparison Operators

In Python, many built-in classes have the following functions that can be used to compare two values:

- `>` : greater than
- `>=` : greater than or equal to
- `<` : less than
- `<=` : less than or equal to
- `==` : equal to
- `!=` : not equal to

Unlike in JavaScript, the `==` function in Python **will not coerce strings to numbers** before comparing them, or perform some of the other type coercions that JavaScript does. For example, in JavaScript, using the `==` operator can lead to some strange behavior:

```
"1" == 1
// => true
0 == []
// => true
[] == ![]
// => true 🤔
```

In Python, the `==` function checks if the objects on both sides are considered the equivalent values:

```
"1" == 1
# False
1 == 1
# True
```

There are some differences between Python's `==` and JavaScript's `===` though. In JavaScript, the `===` operator checks if both objects have the same identity, i.e. refer to the same space in memory. For example, in JavaScript, this example returns `false` because the two arrays are unique objects in memory:

```
[1, 2, 3] === [1, 2, 3];
// => false
```

In Python, this example returns `True` because Python considers these to have equivalent values:

```
[1, 2, 3] == [1, 2, 3]
# True
```

Python will also check if an Integer has the equivalent value to a Float, even though they're technically different data types:

```
1.0 == 1
# True
```

Note: While Python does have an operator, `is`, that is similar to JavaScript's `===`, **it is not used the same way as it is in JavaScript**. There are very few scenarios when you want to use the `is` operator in Python; in general, for comparing data, you want to use the `==`. [See here for examples](https://stackoverflow.com/questions/21774629/python-is-vs-javascript)  [_https://stackoverflow.com/questions/21774629/python-is-vs-javascript_](https://stackoverflow.com/questions/21774629/python-is-vs-javascript) if you're curious about what this operator does.

Logical Operators

Python has the same logical operators you'll find in many other languages, including JavaScript:

- **and** : **and**. Returns **True** if both statements are true.
- **or** : **or**. Returns **True** if one of the two statements is true.
- **not** : **not**. Coerces the data to its boolean equivalent, then reverses it (**True** becomes **False** , and vice versa).

```
True and True
# True
False and False
# False
False and True
# False
True or True
# True
False or False
# False
False or True
# True
not True
# False
not not True
# True
```

Conclusion

In the coming lessons, we'll be writing some functions that use control flow, so make sure to keep these operators for comparing data in mind — they'll be very important to your ability to write conditional logic and looping code successfully!

Resources

- [Python equality](https://realpython.com/courses/python-is-identity-vs-equality/)  (<https://realpython.com/courses/python-is-identity-vs-equality/>).

- [Python operators](https://www.geeksforgeeks.org/python-operators/)  [\(https://www.geeksforgeeks.org/python-operators/\)](https://www.geeksforgeeks.org/python-operators/)